

L'astrazione è una proprietà che consiste nel prendere in considerazione solo gli aspetti più importanti degli oggetti in funzione dell'applicazione che si intende realizzare.

Tipi principali di C++:

- int [4 byte]
- double [8 byte]
- char [1 byte]
- bool [1 byte]
- float [4 byte]

Per dichiarare e inizializzare una variabile:

`tipo nomvar = valore;`

`Es int eta = 20;`

Operatori aritmetici:

- + [somma]
- - [sottrazione]
- * [moltiplicazione]
- / [divisione]
- % [modulo]
- ++ [incremento]
- -- [decremento]

Operatori logici:

- && [and]
- || [or]
- ! [not]

Operatori di confronto:

- == [uguale]
- != [diverso]
- > [maggiore]
- >= [maggiore uguale]
- < [minore]
- <= [minore uguale]

Per stampare messaggi sullo schermo:

`std::cout << ;`

Per leggere quello che l'utente scrive:

`std::cin >> ;`

Per venirti più comodo scrivi questo:

`using namespace std;`

Un namespace è un blocco che contiene definizioni di funzioni e variabili

```
Es using namespace std;
int main () {
    int eta;
    cout << "Quanti anni hai?";
    cin << eta;
}
```

L'if else permette di eseguire codice condizionalmente in base a una espressione booleana:

```
if (condizione) {
    "
} else {
    "
}
```

```
Es if (eta >= 18) {
    cout << "Sei maggiorenne!";
} else {
    cout << "Sei minorenne!";
}
```

Il for è ideale quando si conosce il numero di iterazioni:

```
for (inizializzazione; condizione; incremento) {
    "
}
```

```
Es for(int i=0; i<= 6; i++){
    cout << i << " ";
}
```

Il while si usa quando non sappiamo il numero delle iterazioni:

```
while (condizione) {
    "
}
```

```
Es while (energia > 0) {
    cout << "Sto correndo...";
}
```

Il do-while esegue almeno una volta il ciclo prima di controllare:

```
do {
    "
} while (condizione);
```

```
Es do {
    cout << "Inserisci un numero positivo";
    cin >> numero;
} while (numero <= 0);
```

Una funzione è un blocco di codice riutilizzabile che esegue un compito specifico:

```
tipo_ritorno nomefunzione (parametri) {  
    "  
}
```

Il passaggio dei parametri può avvenire:

- Per valore:

```
void funzione (int x) {}
```

- Per riferimento:

```
void funzione (int& x) {}
```

- Per puntatore:

```
void funzione (int* x) {}
```

La keyword inline chiede al compilatore di sostituire la chiamata direttamente con il corpo della funzione nel punto in cui viene invocata:

```
Es inline int max (int a, int b) {  
    Return (a > b) ? a : b;  
}
```

Un puntatore è una variabile il cui valore non è un dato comune, ma un indirizzo di memoria:

```
tipo *puntatore = &valore;
```

*= accede al valore puntato &= ottiene l'indirizzo di una variabile

```
Es int x= 10;  
    int* p= &x;
```

Un array è una collezione di elementi dello stesso tipo, memorizzati in posizioni di memorie contigue:

```
tipo nomearray [n];
```

Gli array possono avere più dimensioni:

```
tipo nomearray [n] [m];
```

Per usare le stringhe, devi mettere questa libreria:

```
#include <string>
```

Per crearla:

```
string valorestring;
```

```
Es string nome = "Mario" / string nome;
```

Metodi comuni della classe string:

- length ()/ size() [ritorna la lunghezza]
- substr(pos, len) [estrae una sottostringa]
- find (str) [cerca una sottostringa]
- append (str) [aggiunge alla fine]
- insert (pos, str) [inserisce in posizione]
- erase (pos, len) [rimuove caratteri]

- compare (str) [confronta stringhe]

Una classe è un modello per creare oggetti. Raggruppa dati e funzioni che lavorano su quei dati:

```
class nomeclasse {};
```

```
Es class Persona {  
    private:  
    string nome;  
    int eta;  
    public:  
    Persona (string nome, int eta) : nome(nome), eta(eta) {}  
};
```

L'incapsulamento nasconde i dettagli interni di una classe, si realizza tramite modificatori di accesso:

- Private [solo all'interno della classe]
- Public [ovunque]
- Protected [classe e classi derivate]

Costruttore è una funzione speciale chiamata quando si crea un oggetto:

```
Es Persona (string nome, int eta) : nome(nome), eta(eta) {}
```

Distruttore è una funzione chiamata quando l'oggetto viene distrutto:

```
Es ~Persona () { /* pulizia risorse */ }
```

Per accedere e modificare attributi privati, si usa getter e setter:

```
tipo getName() {return attributo; }
```

```
void setName(tipo valore) {controllo -> attributo = valore;}
```

L'ereditarietà permette a una classe di derivare proprietà e metodi da un'altra classe:

```
class Animale {  
    protected:  
    string nome;  
    public:  
    void mangia () {cout << "Sto mangiando";}  
};
```

```
class Cane : public Animale {  
    public:  
    void abbaiare () {cout << "Bau!" << endl;}  
};
```

Esistono 3 tipi di ereditarietà:

- Public [membri public della base restano public]
- Protected [membri public della base diventano protected]
- Private [membri public della base diventano private]

Le funzioni virtuali permettono il polimorfismo, ovvero la proprietà di una funzione di comportarsi in modo differente in funzione dell'oggetto su cui si applica:

```
virtual funzione() {}
```

Una classe astratta contiene almeno una funzione virtuale pura e non può essere istanziata:

```
class nomeclasse {  
    public:  
    virtual funzione () {}  
};
```

Una funzione o classe esterna dichiarata friend può accedere agli attributi privati di una classe, come se fosse dentro la classe stessa:

```
friend tipo nomeFunzione (parametri); / friend class Nomeclasse {};
```

I template permettono di scrivere codice generico che funziona con diversi tipi di dati:

```
template <typename T>
```

new alloca memoria nell'heap a runtime e restituisce un puntatore:

```
tipo* puntatore = new tipo (parametri);
```

delete libera quella memoria quando non serve più:

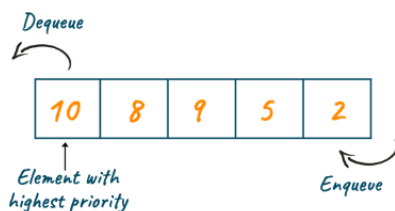
```
delete puntatore;
```

Per gli array:

```
tipo* arr= new tipo [dimensione];  
delete[] arr;
```

Un memory leak si verifica quando la memoria allocata non viene deallocata.

Una coda è una struttura dati che segue il principio fifo



```
template <typename T>
```

```
class Nodo{  
    public:  
    T valore;  
    Nodo* prossimo;  
  
    Nodo(T valore): valore(valore), prossimo(nullptr){}  
};
```

```

template <typename T>
class Coda{
private:
    Nodo<T>* testa=nullptr;
    Nodo<T>* fondo=nullptr;

public:
    Coda(): testa(nullptr), fondo(nullptr){}

    void enqueue(T valore){
        Nodo<T>* nuovo= new Nodo <T> (valore);
        if(!fondo){
            testa=fondo=nuovo;
        }
        fondo->prossimo=nuovo;
        fondo=nuovo;
    }

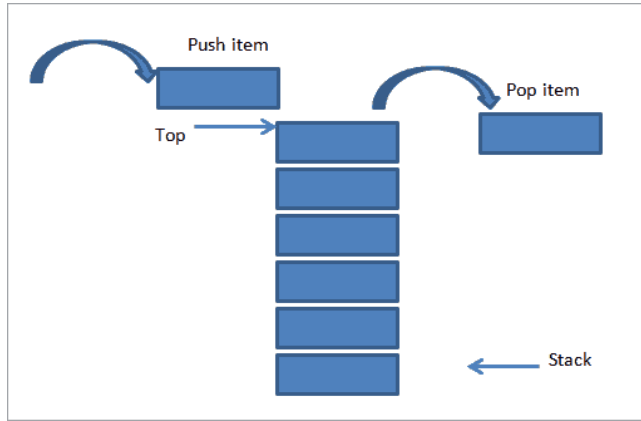
    void dequeue(){
        if(!testa){
            cout << "Coda vuota\n";
            return;
        } else {
            Nodo<T>* tmp= testa;
            testa= testa->prossimo;
            if(!testa) fondo=nullptr;
            delete tmp;
        }
    }

    void stampa() const {
        Nodo<T>* curr=testa;
        while(curr) {
            curr->valore->stampa();
            cout <<< endl;
            curr= curr->prossimo;
        }
    }

    ~Coda(){
        while(testa){
            Nodo<T>* tmp=testa;
            testa= testa->prossimo;
            delete tmp->valore;
            delete tmp;
        }
    }
};

```

La pila è una struttura dati lifo



```
template <typename T>
class Nodo{
public:
    T valore;
    Nodo* prossimo;

    Nodo(T valore) : prossimo(nullptr) {}
};
```

```
template <typename T>
class Pila{
private:
    Nodo <T>* cima = nullptr;

public:
    Pila() : cima(nullptr) {}

    void push(T valore){
        Nodo <T>* nuovo= new Nodo <T> (valore);
        nuovo->prossimo = cima;
        cima=nuovo;
    }

    void pop() {
        if(!cima) {
            cout << "Pila vuota\n";
            return;
        } else {
            Nodo <T>* tmp= cima;
            cima = cima->prossimo;
            delete tmp;
        }
    }
};
```

```
T top() const {
    if(!cima) {
        cout << "Pila vuota\n";
```

```

    }
    return cima->valore;
}

bool empty() const {
    return cima == nullptr;
}

void stampa() const {
    Nodo <T>* curr = cima;

    while(curr) {
        cout << curr->valore << " ";
        curr = curr->prossimo;
    }
    cout << endl;
}

~Pila() {
    while(cima){
        Nodo <T>* tmp = cima;
        cima = cima->prossimo;
        delete tmp;
    }
}
};

```

Lista semplice:

```

template <typename T>
class Nodo{
public:
    T valore;
    Nodo* prossimo;

    Nodo(T valore) : prossimo(nullptr) {}
};

template <typename T>
class Lista {
private:
    Nodo<T>* testa;

public:
    Lista() : testa(nullptr) {}

    void inserisciTesta(T valore) {
        Nodo<T>* nuovo = new Nodo <T> (valore);
        nuovo->prossimo = testa;
    }
};

```



```

    testa = nuovo;
}

void inserisciCoda(T valore) {
    Nodo<T>* nuovo = new Nodo <T> (valore);

    if (testa == nullptr) {
        testa = nuovo;
        return;
    }

    Nodo<T>* curr = testa;
    while (curr->prossimo != nullptr){
        curr = curr->prossimo;
    }
    curr->prossimo = nuovo;
}

void elimina(T valore) {
    if (testa == nullptr) return;

    if (testa->valore == valore) {
        Nodo<T>* tmp = testa->prossimo;
        delete testa;
        testa = tmp;
        return;
    }

    Nodo<T>* curr = testa;
    while (curr->prossimo != nullptr) {
        if (curr->prossimo->valore == valore) {
            Nodo<T>* tmp = curr->prossimo->prossimo;
            delete curr->prossimo;
            curr->prossimo = tmp;
            return;
        }
        curr = curr->prossimo;
    }
}

void stampa() {
    Nodo<T>* curr = testa;
    while (curr != nullptr) {
        cout << curr->valore << " -> ";
        curr = curr->prossimo;
    }
    cout << "nullptr" << endl;
}

```

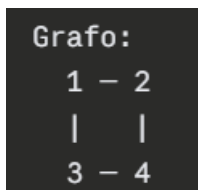
```

~Lista() {
    Nodo<T>* corrente = testa;
    while (corrente != nullptr) {
        Nodo<T>* tmp = curr->prossimo;
        delete curr;
        curr = tmp;
    }
}
};

```

Un grafo è una struttura dati composta da:

- Vertici (o nodi) [gli elementi del grafo]
- Archi (o edge) [la connessione tra i vertici]

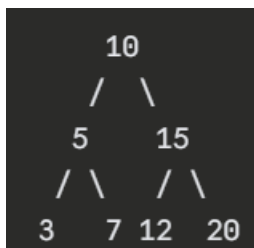


Tipi di grafi:

- Orientato [gli archi hanno una direzione]
- Non orientato [gli archi sono bidirezionali]
- Pesato [ogni arco ha un peso/costo associato]
- Non pesato [tutti gli archi valgono uguali]

Un albero è un tipo speciale di grafo con queste caratteristiche:

- È connesso
- È aciclico
- Ha esattamente n-1 archi per n nodi
- Esiste un unico percorso tra due nodi qualsiasi



Esistono due tipi di visite:

- DFS [visita in profondità]:
- In order (sinistro -> radice -> destro):

3 -> 5 -> 7 -> 10 -> 12 -> 15 -> 20

- Pre order (radice -> sinistro -> destro):

10 -> 5 -> 3 -> 7 -> 15 -> 12 -> 20

- Post order (sinistro -> destro -> radice):

3 -> 7 -> 5 -> 12 -> 20 -> 15 -> 10

- BFS [visita per livelli]

Usi il tipo DFS per:

- Esplorare tutti i percorsi possibili
- L'albero è molto largo
- Problemi di backtracking

Usi il tipo BFS per:

- Trovare il cammino più corto
- Trovare un nodo vicino alla radice
- L'albero è molto profondo

Nel DFS c'è una tecnica dove i nodi vengono colorati durante la visita per sapere il loro stato:

- Bianco [nodo non ancora visitato]
- Grigio [nodo in visita]
- Nero [nodo completamente visitato]

Codice base per costruire gli alberi:

```
template <typename T>
class Nodo {
    public:
        T valore;
        Nodo<T>* sinistro;
        Nodo<T>* destro;

        Nodo(T valore) : valore(valore), sinistro(nullptr), destro(nullptr) {}
};
```

```
template <typename T>
class Albero {
    public:
        Nodo<T>* root;

        Albero() : root(nullptr) {}
};
```

Vista DFS da implementare dentro l'albero:

```
void inorder(Nodo<T>* nodo) {
    if(nodo==nullptr) return;

    inorder(nodo->sinistro);
    cout << nodo->valore << " ";
    inorder(nodo->destro);
}
```

```
void preorder(Nodo<T>* nodo) {
    if(nodo==nullptr) return;

    cout << nodo->valore << " ";
```

```

        preorder(nodo->sinistro);
        preorder(nodo->destro);
    }

void postorder(Nodo<T>* nodo) {
    if(nodo==nullptr) return;

    postorder(nodo->sinistro);
    postorder(nodo->destro);
    cout << nodo->valore << " ";
}

```

Con il BFS l'inserimento si fa così:

```

Nodo<T>* inserisci(Nodo<T>* nodo, T valore){
    if(nodo==nullptr) {
        return new Nodo<T> (valore);
    }

    if(valore < nodo->valore) {
        nodo->sinistro = inserisci(nodo->sinistro, valore);
    } else if (valore > nodo->valore) {
        nodo->destro = inserisci(nodo->destro, valore);
    }

    return nodo
}

void inserisci(T valore) {
    root = inserisci(root, valore);
}

```

Per la ricerca in BFS scriviamo così:

```

Nodo<T>* ricerca(Nodo<T>* nodo, T valore) {
    if(nodo == nullptr || nodo->valore == valore) {
        return nodo;
    }

    if(valore < nodo->valore) {
        return cerca(nodo->sinistro, valore);
    } else {
        return cerca(nodo->destro, valore);
    }
}

Bool cerca (T valore) {
    return cerca(root, valore) != nullptr;
}

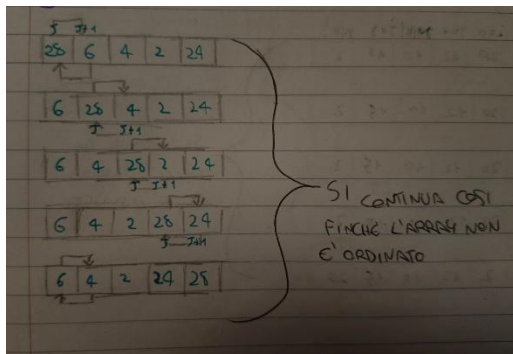
```

Bubble sort:

Confronta elementi adiacenti e li scambia se sono nell'ordine sbagliato.

Complessità temporale: $O(n^2)$ nel caso peggiore e medio mentre $O(n)$ nel caso migliore.

```
void bubbleSort(int arr[], int n) {  
    for(int i=0; i<n-1; i++) {  
        for(int j=0; j<n-i-1;j++) {  
            if(arr[j] > arr[j+1]){  
                int tmp=arr[j];  
                arr[j] = arr[j+1];  
                arr[j+1]= tmp;  
            }  
        }  
    }  
}
```

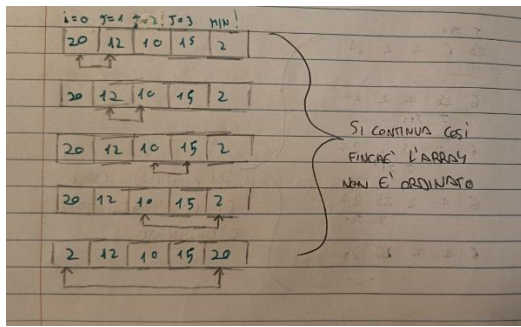


Selection sort:

Trova l'elemento minimo e lo sposta all'inizio, poi ripete per il resto dell'array.

Complessità temporale: $O(n^2)$ in tutti i casi

```
void selectionSort(int arr[], int n) {  
    for (int i = 0; i < n - 1; i++) {  
        int minIndex = i;  
  
        for (int j = i + 1; j < n; j++) {  
            if (arr[j] < arr[minIndex])  
                minIndex = j;  
        }  
        int temp = arr[i];  
        arr[i] = arr[minIndex];  
        arr[minIndex] = temp;  
    }  
}
```

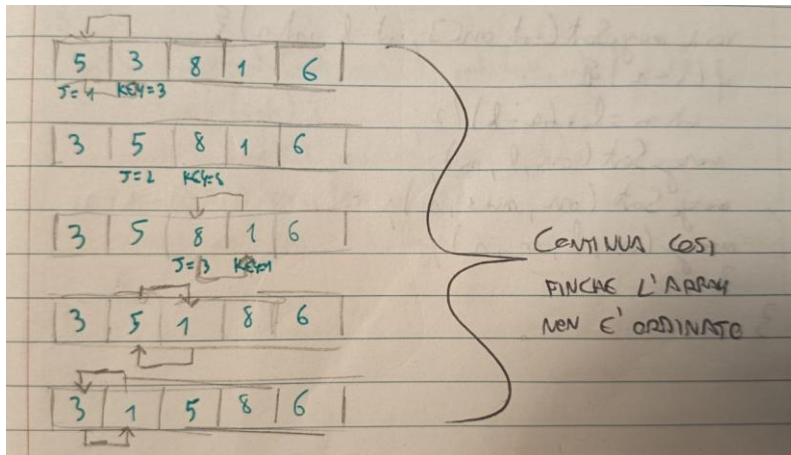


Insertion sort:

Costruisce l'array ordinato un elemento alla volta, inserendo ogni elemento nella posizione corretta.

Complessità temporale: $O(n^2)$ nel caso peggiore mentre $O(n)$ nel caso migliore

```
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        for (int j = i; j > 0; j--) {
            if (arr[j] < arr[j - 1]) {
                int temp = arr[j];
                arr[j] = arr[j - 1];
                arr[j - 1] = temp;
            } else {
                break;
            }
        }
    }
}
```



Merge sort:

Algoritmo dividi-et-impera che divide l'array a metà, ordina ricorsivamente le due metà e le unisce.

Complessità temporale: $O(n \log n)$ in tutti i casi

```
void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1;
    int n2 = r - m;

    int L[n1], R[n2];
```

```
for (int i = 0; i < n1; i++) L[i] = arr[l + i];
for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
```

```
int i = 0, j = 0, k = l;
```

```
while (i < n1 && j < n2) {
    arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
}
```

```
while (i < n1) arr[k++] = L[i++];
while (j < n2) arr[k++] = R[j++];
```

```
}
```

```
void mergeSort(int arr[], int l, int n) {
    if (l < n) {
        int m = l + (n - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, n);
        merge(arr, l, m, n);
    }
}
```

Quick sort:

Algoritmo divide-et-impera che sceglie un pivot e partiziona l'array.

Complessità temporale: $O(n \log n)$ in media, $O(n^2)$ nel caso peggiore

```
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            int tmp = arr[i];
            arr[i] = arr[j];
            arr[j] = tmp;
        }
    }

    int tmp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = tmp;

    return i + 1;
}
```

```
void quickSort(int arr[], int low, int high) {
```

```
if (low < high) {  
    int pi = partition(arr, low, high);  
    quickSort(arr, low, pi - 1);  
    quickSort(arr, pi + 1, high);  
}  
}
```

Un cast serve a convertire un tipo di dato in un altro

Ci sono 4 tipi principali:

- static_cast [conversioni normali]

```
int a = 5;
```

```
double b = static_cast <double> (a);
```

- dynamic_cast [classi(ereditarietà)]

```
Base* b = new Derivata();
```

```
Derivata* d = dynamic_cast <Derivata*> (b);
```

- const_cast [serve per aggiungere o rimuovere const]

```
Const int a = 10;
```

```
int* p = const_cast <int*> (&a);
```

- reinterpret_cast [serve per reintepare i bit in memoria]